27

BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

Executors Fabrika Tuzakları: newFixedThreadPool Neden Üretimde OOM Yaratır?

Executors Factory Traps: Why newFixedThreadPool Triggers Production OOM

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              EXECUTORS.NEWFIXEDTHREADPOOL() ÖLÜMCÜL KODU     │
└──────────────────────────────────────────────────────────────┘

Executors.newFixedThreadPool(10) ÇAĞRISININ ARKA PLANI:

return new ThreadPoolExecutor(
    10, 10, 0L, TimeUnit.MILLISECONDS,
    new LinkedBlockingQueue<Runnable>() // 💣 SAATLİ BOMBA BURADA!
);

LinkedBlockingQueue'nun Varsayılan Kapasitesi:
Integer.MAX_VALUE = 2,147,483,647 (2 MİLYAR GÖREV!)

GÖREVLER HAVUZA AKARKEN:
10 Core Thread kilitlenir veya yavaşlar ──► Gelen görevler kuyrukta birikir:
[Task 1, Task 2, Task 3, ... Task 1,000,000, ... Task 5,000,000]
                                        │
                                        ▼
HEAP BELLEK BİTER: java.lang.OutOfMemoryError: Java heap space ──► SİSTEM ÇÖKER!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir mağaza açıyorsunuz. İçeriye 10 tezgahar koyuyorsunuz (newFixedThreadPool(10)). Ama bekleme salonunu 'sonsuz' büyüklükte yapıyorsunuz (2 milyar kişilik kuyruk)! Tezgahlarda işler biraz yavaşladığında kapıdaki 5 milyon insan mağazanın koridorlarına doluşur. Sonunda binanın kolonları bu ağırlığı taşıyamaz ve bina herkesin üstüne çöker (OutOfMemoryError)! İşte `Executors.newFixedThreadPool` kullanmak, mağazanın kapısına sınırsız insan alıp binayı çökertmektir!

🇬🇧 ENGLISH (ELI5)

Imagine opening a bank branch with 10 tellers (`newFixedThreadPool(10)`). However, you build a waiting lobby capable of holding 2.14 billion humans (`LinkedBlockingQueue`). If the tellers suffer an external network delay, millions of customers flood into the building. Eventually, the floor collapses under the physical weight, demolishing the entire bank (`OutOfMemoryError`). Never configure an infinite lobby!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Executors Metotlarının MAANG Üretim Analizi: 1) newFixedThreadPool & newSingleThreadExecutor Felaketi: İkisi de unbounded `LinkedBlockingQueue` (kapasite: 2^31-1) kullanır. Tüketici hızı üretici hızından yavaş olduğunda görev nesneleri Heap'i doldurur ve JVM OOM ile çöker. 2) newCachedThreadPool Felaketi: `corePoolSize = 0`, `maximumPoolSize = Integer.MAX_VALUE` ve `SynchronousQueue` kullanır. Yoğun trafikte her gelen istek için yeni bir OS thread'i açmaya çalışır; binlerce thread açıldığında işletim sistemi seviyesinde `java.lang.OutOfMemoryError: unable to create new native thread` patlar. 3) MAANG Kuralı: `Executors` statik fabrikalarının kullanımı kurumsal kod tabanlarında kesinlikle yasaklanmalıdır (Checkstyle / SonarQube kuralı); daima parametreleri açıkça belirlenmiş `ThreadPoolExecutor` kurulmalıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Why Static Executors Factories are Banned at MAANG: 1) `newFixedThreadPool` Unbounded Queue Doom: It wraps an unbounded `LinkedBlockingQueue` (capacity `Integer.MAX_VALUE`). Under downstream latency spikes, tasks accumulate on heap memory until a catastrophic `OutOfMemoryError: Java heap space` kills the JVM process. 2) `newCachedThreadPool` Thread Explosion Doom: It sets `maximumPoolSize = Integer.MAX_VALUE` over a zero-capacity `SynchronousQueue`. In traffic surges, it spawns thousands of OS platform threads until memory or OS limits trip, crashing with `OutOfMemoryError: unable to create native thread`. 3) Staff+ Guideline: Enforce linter rules prohibiting `java.util.concurrent.Executors` factory methods. Always instantiate `ThreadPoolExecutor` with explicit bounded queues and custom rejection handlers.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Güvenli Bounded Havuz vs Tehlikeli Executors Karşılaştırması
import java.util.concurrent.*;

public class SafeVsUnsafePools {
    // ❌ TEHLİKELİ (Üretimde KESİNLİKLE kullanmayın!):
    public static ExecutorService createDangerousPool() {
        return Executors.newFixedThreadPool(10); // Unbounded kuyruk = OOM riski!
    }

    // ✔ GÜVENLİ (MAANG Standardı):
    public static ExecutorService createEnterpriseSafePool() {
        return new ThreadPoolExecutor(
            8,                          // Core pool size
            16,                         // Max pool size
            30L, TimeUnit.SECONDS,      // Idle timeout
            new ArrayBlockingQueue<>(5000), // ⚠ Sınırlı Kuyruk (Bounded Queue)
            Thread.ofPlatform().name("safe-worker-", 1).factory(),
            new ThreadPoolExecutor.CallerRunsPolicy() // Backpressure
        );
    }

    void main() {
        var safePool = createEnterpriseSafePool();
        safePool.execute(() -> IO.println("Güvenli kurumsal havuz aktif!"));
        safePool.shutdown();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Klasiği: 'Executors.newCachedThreadPool() ne zaman çöker?' Yanıt: 'Gelen istek hızı thread'lerin işi bitirme hızından yüksek olduğunda, havuz sonsuza kadar yeni OS thread'i açmaya çalışır. 10.000 thread açıldığında sistem RAM'i tükenir ve unable to create new native thread hatasıyla çöker!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Interview Distinction: 'What is the failure mode of `newCachedThreadPool`?' Senior Answer: 'Unbounded thread creation! Because maximumPoolSize is Integer.MAX_VALUE, any traffic burst spawns thousands of native threads. Each consumes 1MB stack plus OS kernel metadata, crashing the JVM with `OutOfMemoryError: unable to create new native thread`.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- newFixedThreadPool sınırsız LinkedBlockingQueue yüzünden Heap OOM yaratır.
- newCachedThreadPool sınırsız thread açtığı için Native Thread OOM patlatır.
- Üretimde DAİMA sınırlı kuyruklu (ArrayBlockingQueue) açık ThreadPoolExecutor kullanılır.

🎯 Key Takeaways for the Interview

- newFixedThreadPool uses an unbounded LinkedBlockingQueue, causing Heap OOM crashes.
- newCachedThreadPool spawns infinite threads, causing Native Thread OOM panics.
- In production, always construct explicit ThreadPoolExecutors with bounded queues.